Chapter :2

What is Searching?

- Searching is the process of finding a given value position in a list of values.
- It decides whether a search key is present in the data or not.
- It is the algorithmic process of finding a particular item in a collection of items.
- It can be done on the internal data structure or on the external data structure.

Searching Techniques

To search an element in a given array, it can be done in the following ways:

1. Sequential Search
2. Binary Search

1. Sequential Search

- Sequential search is also called Linear Search.
- Sequential search starts at the beginning of the list and checks every element of the list.
- It is a basic and simple search algorithm.
- Sequential search compares the element with all the other elements given in the list. If the element is matched, it returns the value index, else it returns -1.

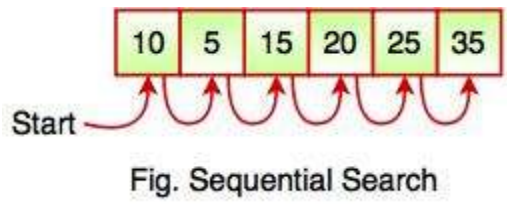


Fig. Sequential Search

The above figure shows how sequential search works. It searches an element or value from an array till the desired element or value is not found. If we search the element 25, it will go step by step in a sequence order. It searches in a sequence order. Sequential search is applied on the unsorted or unordered list when there are fewer elements in a list.

The following code snippet shows the sequential search operation:

Example: Program for Sequential Search

```
#include <stdio.h>
int main()
{
  int num[50], target,count,i,flag=0;
  printf("Enter the number of elements in array\n");
  scanf("%d",&count);
  printf("Enter %d integer(s)\n", count);
  for (i = 0; i < count; i++)
  {
    scanf("%d", &num[i]);
  }
  printf("Enter the number to search\n");
  scanf("%d", &target);
  for (i = 0; i < count; i++)
  {
    if (num[i] == target)     /* if required element found */
    {
      printf("%d is present at location %d.\n", target, i+1);
      flag=1;
      break;
    }
  }
  if (flag==0)
  {
    printf("%d is not present in array.\n", target);
  }
}
```

2. Binary Search

- Binary Search is used for searching an element in a sorted array.
- It is a fast search algorithm with run-time complexity of O(log n).
- Binary search works on the principle of divide and conquer.
- This searching technique looks for a particular element by comparing the middlemost element of the collection.
- It is useful when there are large number of elements in an array.

- The above array is sorted in ascending order. As we know binary search is applied on sorted lists only for fast searching.

For example, if searching an element 25 in the 7-element array, following figure shows how binary search works:

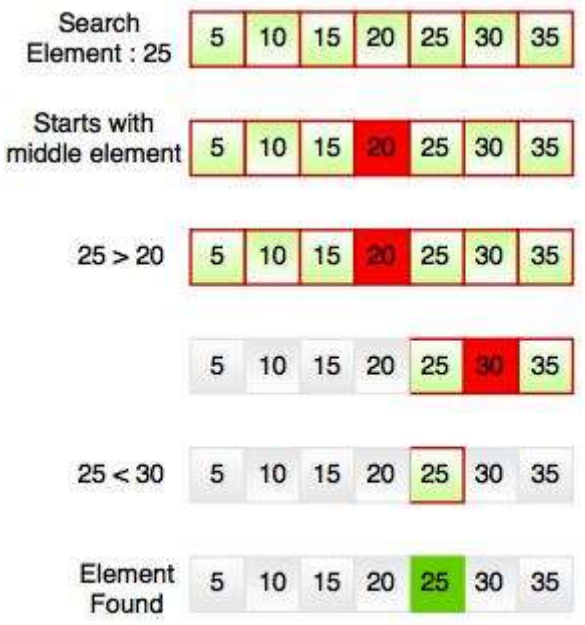


Fig. Working Structure of Binary Search

Binary searching starts with the middle element. If the element is equal to the element that we are searching for, then return true. If the element is less than then move to the right of the list or if the element is greater than then move to the left of the list. Repeat this, till you find an element.

```
#include <stdio.h>
void main()
{
  int num[10]={6,12,18,22,31,39,55,65,74,82};
  int target,mid;
  int start=0,end=9;
  printf("\nEnter your target value=");
  scanf("%d",&target);
  while(start<=end)
  {
    mid=(start+end)/2;
    if(num[mid]==target)
    {
      printf("Element %d found at %d position",target,mid+1);
      break;
    }
    else
    {
      if(num[mid]<target)
      {
```

```
                start=mid+1;
            }
            else
            {
                end=mid-1;
            }
        }
    }
}
```

3. Sorting: -

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order.

The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats. Following are some of the examples of sorting in real-life scenarios −

- **Telephone Directory** − The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- **Dictionary** − The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

3.1 Selection Sort Algorithm:

n selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. It is also the simplest algorithm. It is an in-place comparison sorting algorithm. In this algorithm, the array is divided into two parts, first is sorted part, and another one is the unsorted part. Initially, the sorted part of the array is empty, and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.

The average and worst-case complexity of selection sort is **O(n^2)**, where **n** is the number of items. Due to this, it is not suitable for large data sets.

Selection sort is generally used when -

- A small array is to be sorted
- Swapping cost doesn't matter
- It is compulsory to check all elements

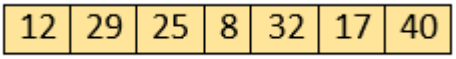
Now, let's see the algorithm of selection sort.

Working of Selection sort Algorithm

Now, let's see the working of the Selection sort Algorithm.

To understand the working of the Selection sort algorithm, let's take an unsorted array. It will be easier to understand the Selection sort via an example.

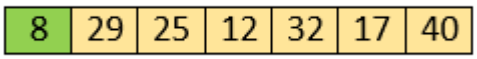
Let the elements of array are -



Now, for the first position in the sorted array, the entire array is to be scanned sequentially.

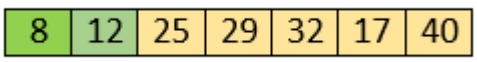
At present, **12** is stored at the first position, after searching the entire array, it is found that **8** is the smallest value.

So, swap 12 with 8. After the first iteration, 8 will appear at the first position in the sorted array.

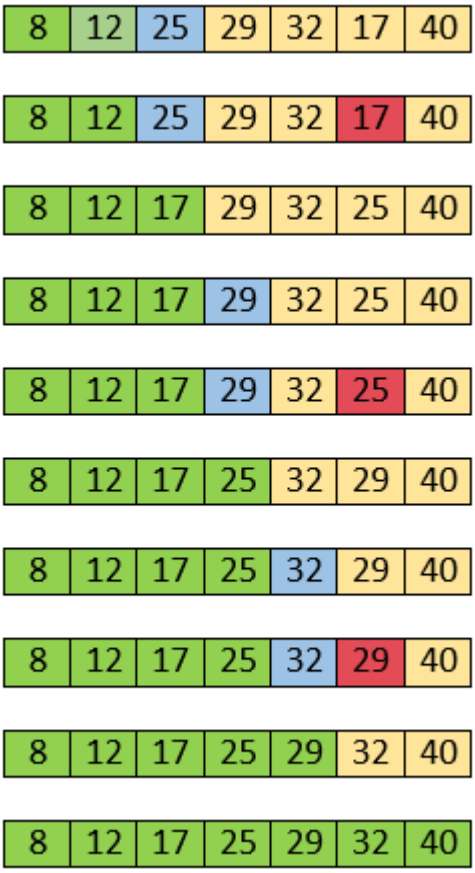


For the second position, where 29 is stored presently, we again sequentially scan the rest of the items of unsorted array. After scanning, we find that 12 is the second lowest element in the array that should be appeared at second position.

Now, swap 29 with 12. After the second iteration, 12 will appear at the second position in the sorted array. So, after two iterations, the two smallest values are placed at the beginning in a sorted way.



The same process is applied to the rest of the array elements. Now, we are showing a pictorial representation of the entire sorting process.



Now, the array is completely sorted.

Selection sort complexity

Now, let's see the time complexity of selection sort in best case, average case, and in worst case. We will also see the space complexity of the selection sort.

1.

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Time Complexity

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of selection sort is **$O(n^2)$**.

- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of selection sort is **$O(n^2)$**.
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of selection sort is **$O(n^2)$**.

2. Space Complexity

Space Complexity	O(1)
Stable	YES

- The space complexity of selection sort is O(1). It is because, in selection sort, an extra variable is required for swapping.

Implementation of selection sort

Now, let's see the programs of selection sort in different programming languages.

Program: **Write a program to implement selection sort in C language**.

```
//WAP in 'C' to implement selection sort
#include<stdio.h>
//#include<conio.h>
void main()
{
   int arr[6]={5,9,7,12,2};
   int i,j,k,temp;
   for(i=0;i<5;i++)
   {
     for(j=i+1;j<5;j++)
    {
        if(arr[i]>arr[j])
       {
            temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
        }
     }
   }
   for(k=0;k<5;k++)
   {
      printf("%d ",arr[k]);
   }
}
```

3.1 Bubble Sort Algorithm:

Bubble sort works on the repeatedly swapping of adjacent elements until they are not in the intended order. It is called bubble sort because the movement of array elements is just like the movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly, the array elements in bubble sort move to the end in each iteration.

Although it is simple to use, it is primarily used as an educational tool because the performance of bubble sort is poor in the real world. It is not suitable for large data sets. The average and worst-case complexity of Bubble sort is **$O(n^2)$,** where **n** is a number of items.

Bubble short is majorly used where -

- complexity does not matter
- simple and shortcode is preferred

Algorithm

In the algorithm given below, suppose **arr** is an array of **n** elements. The assumed **swap** function in the algorithm will swap the values of given array elements.

```
1. begin BubbleSort(arr)
2.   for all array elements
3.     if arr[i] > arr[i+1]
4.       swap(arr[i], arr[i+1])
5.     end if
6.   end for
7.   return arr
8. end BubbleSort
```

Working of Bubble sort Algorithm

Now, let's see the working of Bubble sort Algorithm.

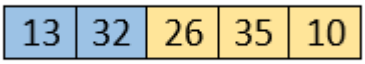
To understand the working of bubble sort algorithm, let's take an unsorted array. We are taking a short and accurate array, as we know the complexity of bubble sort is **$O(n^2)$.**

Let the elements of array are -

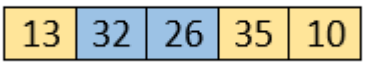
13	32	26	35	10

First Pass

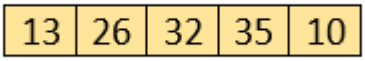
Sorting will start from the initial two elements. Let compare them to check which is greater.



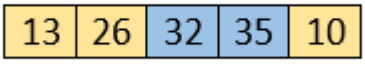
Here, 32 is greater than 13 (32 > 13), so it is already sorted. Now, compare 32 with 26.



Here, 26 is smaller than 36. So, swapping is required. After swapping new array will look like -

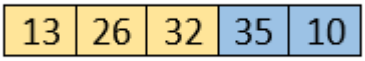


Now, compare 32 and 35.

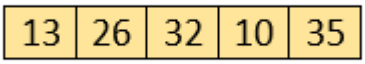


Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.



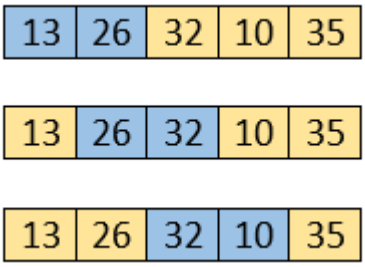
Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be -



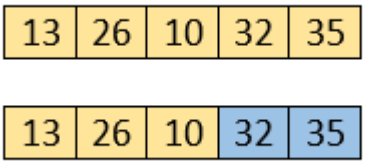
Now, move to the second iteration.

Second Pass

The same process will be followed for second iteration.



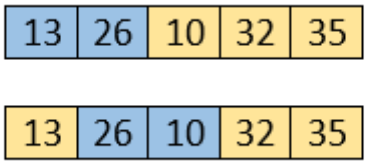
Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be -



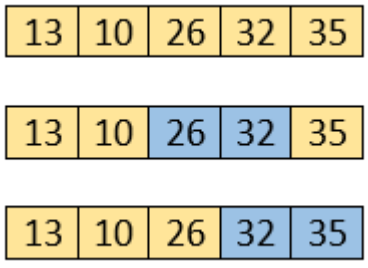
Now, move to the third iteration.

Third Pass

The same process will be followed for third iteration.



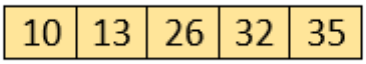
Here, 10 is smaller than 26. So, swapping is required. After swapping, the array will be -



Now, move to the fourth iteration.

Fourth pass

Similarly, after the fourth iteration, the array will be -



Hence, there is no swapping required, so the array is completely sorted.

Bubble sort complexity

Now, let's see the time complexity of bubble sort in the best case, average case, and worst case. We will also see the space complexity of bubble sort.

1. Time Complexity

Case	Time Complexity
Best Case	O(n)
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of bubble sort is **O(n).**
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of bubble sort is **$O(n^2)$.**
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of bubble sort is **$O(n^2)$.**

2. Space Complexity

Space Complexity	O(1)
Stable	YES

- The space complexity of bubble sort is O(1). It is because, in bubble sort, an extra variable is required for swapping.
- The space complexity of optimized bubble sort is O(2). It is because two extra variables are required in the optimized bubble sort.

Now, let's discuss the optimized bubble sort algorithm.

Optimized Bubble Sort Algorithm

In the bubble sort algorithm, comparisons are made even when the array is already sorted. Because of that, the execution time increases.

To solve it, we can use an extra variable ***swapped.*** It is set to **true** if swapping requires; otherwise, it is set to **false.**

It will be helpful, as suppose after an iteration if there is no swapping required, the value of the variable **swapped** will be **false.** It means that the elements are already sorted, and no further iterations are required.

This method will reduce the execution time and also optimizes the bubble sort.

Algorithm for optimized bubble sort

```
1. bubbleSort(array)
2. n = length(array)
3. repeat
4.   swapped = false
5.   for i = 1 to n - 1
6.         if array[i - 1] > array[i], then
7.         swap(array[i - 1], array[i])
8.         swapped = true
9.         end if
10.   end for
11.   n = n - 1
12.  until not swapped
13. end bubbleSort
```

Implementation of Bubble sort

Now, let's see the programs of Bubble sort in different programming languages.

Program: Write a program to implement bubble sort in C language.

```
/WAP in 'C' to implement bubble sort
#include<stdio.h>
//#include<conio.h>
void main()
{
   int arr[6]={5,9,7,12,2,67};
   int i,j,k,temp,xc=0;
   int n=6;
```

```
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-1-i;j++)
        {
          if(arr[j]>arr[j+1])
          {
              temp=arr[j];
              arr[j]=arr[j+1];
              arr[j+1]=temp;
          }
        }
    }
    for(k=0;k<5;k++)
    {
       printf("%d ",arr[k]);
    }
}
```